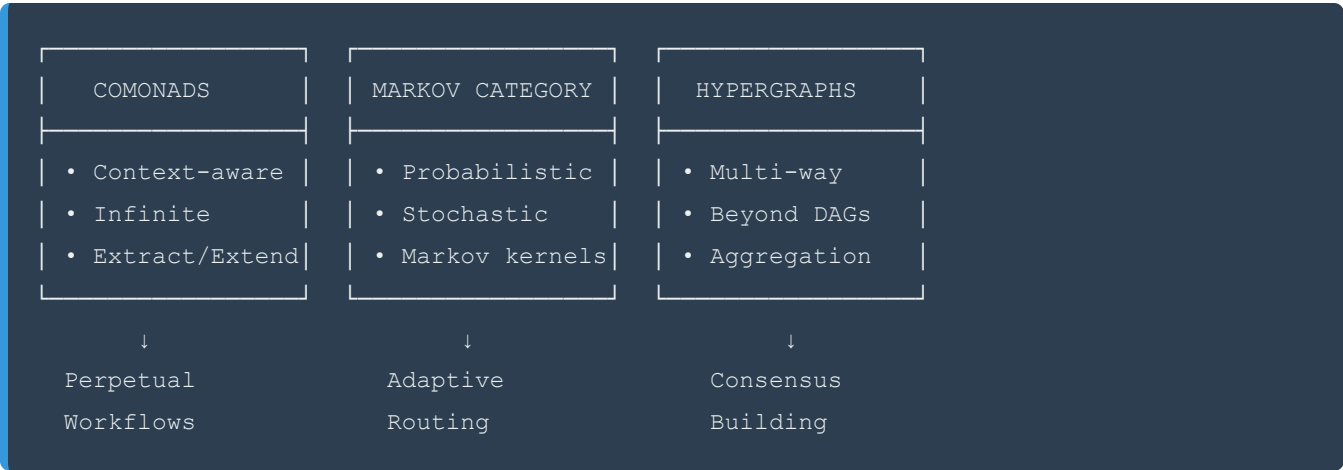


Level 7 Quick Reference Guide

Claude Code Agent Orchestration DSL - Level 7 Cheat Sheet Date: 2025-10-19 ---

Three Pillars at a Glance



Syntax Quick Reference

1. Comonadic Context

```
// Define comonad
comonad LLMContext {
  extract(ctx) = ctx.focus
  duplicate(ctx) = ctx { focus = ctx }
  extend(f, ctx) = ctx { focus = f(ctx) }
}

// Perpetual loop
workflow infinite {
  extend(self_refine) : initial → Result^∞
}

// Context-aware agent
agent researcher[ctx: LLMContext] {
  // Automatic context access
  output = generate(ctx.focus, ctx.history, ctx.temperature)
}
```

2. Probabilistic Morphisms

```
// Markov kernel
P(output | input) : Input → Dist(Output)

// Stochastic composition
workflow probabilistic {
  input → P(classify) → P(route) → P(execute)
}

// Conditional probabilities
P(agent | difficulty) = {
  easy: {gpt35: 0.8, gpt4: 0.2}
  hard: {gpt35: 0.1, gpt4: 0.9}
}

// Bayesian update
update_posterior(agent, result.quality)
```

3. Hypergraph Dependencies

```
// Multi-way edge
hypergraph consensus {
  // Vertex syntax
  vertices: [A, B, C, D]

  // Hyperedge syntax: ({sources}, {targets})
  edge e1 = ({A, B, C}, {D})
    aggregation: "consensus"
    probability: 0.8
}

// Aggregation strategies
aggregation = {
  "concat" | "consensus" | "weighted_vote" |
  "embedding_mean" | "attention"
}
```

Common Patterns

Pattern 1: Self-Refining Agent

```
workflow self_refining(query) {  
  ctx = LLMContext {  
    system_prompt: "You are an expert",  
    history: [],  
    temperature: 0.7,  
    focus: query  
  }  
  
  stream = extend(  
    λ(c) → {  
      response = llm_call(c)  
      quality = evaluate(response)  
  
      if quality > 0.9:  
        return response  
      else:  
        return refine(c)  
    },  
    ctx  
  )  
  
  return stream.take_until(satisfied)  
}
```

Pattern 2: Difficulty-Aware Router

```
workflow adaptive_router(query) {  
  // Estimate difficulty (stochastic)  
  difficulty ~ estimate_difficulty(query)  
  
  // Route based on probability  
  model ~ P(model | query, difficulty)  
  
  // Execute  
  result = execute(model, query)  
  
  // Learn (Thompson Sampling)  
  update_posterior(model, result)  
  
  return result  
}
```

Pattern 3: Multi-Agent Consensus

```
hypergraph research {  
  vertices: [researcher_A, researcher_B, researcher_C, synthesizer]  
  
  // Multi-way: synthesizer needs ALL researchers  
  edge e1 = ({researcher_A, researcher_B, researcher_C}, {synthesizer})  
    aggregation: "weighted_vote"  
    weights: {A: 0.4, B: 0.35, C: 0.25}  
  
  // Probabilistic next step  
  edge e2 = ({synthesizer}, {validator}) probability: 0.7  
  edge e3 = ({synthesizer}, {refine}) probability: 0.3  
}
```

Mathematics Cheat Sheet

Comonad Operations

```
extract : W a → a           (get focused value)
duplicate : W a → W (W a)    (nest context)
extend : (W a → b) → W a → W b  (apply with context)
```

Laws:

```
extract ∘ duplicate = id
fmap extract ∘ duplicate = id
fmap duplicate ∘ duplicate = duplicate ∘ duplicate
```

Markov Kernels

```
k: X ×  $\Sigma_Y$  → [0,1]           (stochastic map)
( $\ell \circ k$ ) (C|x) =  $\int_Y \ell(C|y) k(dy|x)$   (composition)
```

Kleisli:

```
Stoch(X,Y)  $\cong$  Meas(X, P(Y))
```

Hypergraph Structures

```
H = (V, E) where  $e \in E \subseteq P(V)$       (undirected)
DH:  $e = (T(e), H(e))$                      (directed)
PH:  $M[i,j] \in [0,1]$                      (probabilistic)
```

```
Topological sort:  $O(|V| + \sum_e (|T| + |H|))$ 
```

Complexity Quick Reference

| Operation | Complexity | Notes | |-----|-----|-----| | Comonadic extend | $O(1)$ per iteration | Lazy evaluation | | Markov composition | $O(n^2) \rightarrow O(n)$ | Sparse kernels | | Hypergraph toposort | $O(|V| + |E|)$ | Extended algorithm | | Expected makespan | **#P-hard** | Use Monte Carlo | | Tensor contraction | $O(n^k) \rightarrow O(R \cdot n \cdot k)$ | CP decomposition | | Bayesian optimization | $O(N^3) \rightarrow O(N^2)$ | Sparse GP | ---

Token/Time Budgets

Workflow Type	Tokens	Time
Comonadic Perpetual	∞ (stream)	continuous
Probabilistic Router	13K-53K	2.5-10.5 min
Hypergraph Multi-Way	46K-90K	~9 min
Self-Optimizing	230K-430K	2-4 hours
Tensor Network	$5K + 2K \cdot n$	setup + exec

When to Use Level 7

✔ Use Level 7 For:

- 1. **Perpetual workflows** (infinite loops, streaming)
- 2. **Probabilistic routing** (adaptive model selection)
- 3. **Multi-agent consensus** (3+ agents synchronized)
- 4. **Self-optimization** (learning workflows)
- 5. **Research problems** (frontier applications)

✖ Avoid Level 7 For:

- 1. **Simple tasks** → Use Levels 1-3
 - 2. **Deterministic** → Use Levels 4-5
 - 3. **Pairwise only** → Use Level 6
 - 4. **Low overhead** → Lower levels
 - 5. **Need debugging** → Simpler models
-

Common Gotchas

1. Infinite Loops

```
// ❌ BAD: No termination
extend(refine) : ctx → Context∞

// ✅ GOOD: With termination
extend(refine) : ctx → Context∞
  .take_until(converged)
```

2. Probability Normalization

```
// ❌ BAD: Probabilities don't sum to 1
P(A) = 0.6, P(B) = 0.7 // Total = 1.3!

// ✅ GOOD: Normalized
P(A) = 0.6, P(B) = 0.4 // Total = 1.0
```

3. Hypergraph Cycles

```
// ❌ BAD: Creates cycle
edge e1 = ({A}, {B})
edge e2 = ({B}, {A}) // Cycle!

// ✅ GOOD: Acyclic (DAH)
edge e1 = ({A}, {B})
edge e2 = ({B}, {C})
```

Execution Flow Shortcuts

Comonadic Loop

```
W0 →{extend f} W1 →{extend f} W2 → ... → W∞
```

Probabilistic Sampling

1. Sample active edges: if random() < P(e): activate
2. Topological sort
3. Execute in order
4. Aggregate results

Hypergraph Multi-Way

1. Wait for ALL sources in T(e)
2. Aggregate contexts
3. Execute target
4. Distribute to heads H(e)

DSL Operators Summary

Level 7 Additions

Operator	Syntax	Meaning
<code>extend</code>	<code>extend(f, ctx)</code>	Apply f with full context
<code>extract</code>	<code>extract(ctx)</code>	Get focused value
<code>duplicate</code>	<code>duplicate(ctx)</code>	Nest context
<code>~</code>	<code>x ~ Dist</code>	Sample from distribution
<code>P(·\ ·)</code>		Conditional probability
<code>{·, ·}</code>		Hyperedge vertex set
<code>→</code>		Directed hyperedge

Aggregation Functions

```
aggregation = {  
  concat:      "A | B | C"  
  consensus:   majority_vote([A, B, C])  
  weighted_vote:  $\sum w_i \cdot A_i$   
  embedding_mean: mean([embed(A), embed(B), embed(C)])  
  attention:   cross_attention(query, [A, B, C])  
}
```

Optimization Strategies

1. Path Pruning

```
prune_paths(threshold = 0.01) {  
  for each path P:  
    if prob(P) < threshold:  
      exclude P  
  renormalize remaining paths  
}
```

2. Bayesian Optimization

```
optimize_workflow {  
  params ~ GaussianProcess_posterior  
  
  loop:  
     $\theta_{\text{next}} = \operatorname{argmax} EI(\theta)$   
    result = execute_workflow( $\theta_{\text{next}}$ )  
    observe( $\theta_{\text{next}}$ , result)  
    update_posterior( $\theta_{\text{next}}$ , result)  
  
    if converged: break  
}
```

3. Tensor Decomposition

```
// Original:  $O(n^k)$   
 $T[i,j,k] \in \mathbb{R}^{n \times n \times n}$   
  
// Decomposed:  $O(R \cdot n \cdot k)$   
 $T \approx \sum_r \lambda_r \cdot a^1_r \otimes a^2_r \otimes a^3_r$   
  
// Typical:  $R=10, n=100 \rightarrow 100\times$  speedup
```

Error Handling

```
workflow with_fallback {
  try:
    primary_path
  catch(error):
    if retries < max_retries:
      retry(primary_path)
    else:
      fallback_path
}

// Probabilistic fallback
workflow probabilistic_fallback {
  edge e1 = ({A}, {B}) probability: 0.8
  edge e2 = ({A}, {C}) probability: 0.2 // fallback

  execute with sampling
}
```

Testing Level 7 Workflows

Unit Tests

```
def test_comonad_laws():
    ctx = LLMContext(focus="test")

    # Left counit
    assert extract(duplicate(ctx)) == ctx

    # Right counit
    assert fmap(extract, duplicate(ctx)) == ctx

    # Coassociativity
    assert fmap(duplicate, duplicate(ctx)) == duplicate(duplicate(ctx))
```

Integration Tests

```
def test_hypergraph_execution():
    H = create_hypergraph(
        vertices=[A, B, C, D],
        edges=[({A, B, C}, {D})]
    )

    result = execute_hypergraph(H, initial_input)

    assert result.D.depends_on([A, B, C])
    assert result.aggregation_called
```

Property-Based Tests

```
@given(hypergraph=st.hypergraphs(),
       input_data=st.data())
def test_topological_sort_consistency(hypergraph, input_data):
    order = topological_sort(hypergraph)

    # Property: all edges respect ordering
    for edge in hypergraph.edges:
        for tail in edge.tails:
            for head in edge.heads:
                assert order.index(tail) < order.index(head)
```

Performance Tips

1. Lazy Evaluation

```
// Use lazy streams for infinite workflows
stream = extend(f, ctx).lazy()
result = stream.take(n) // Only compute n items
```

2. Parallel Execution

```
// Hypergraph enables natural parallelism
edge e = ({A, B, C}, {D})
  parallel: true // Execute A, B, C in parallel
```

3. Caching

```
// Cache deterministic computations
agent expensive_computation[cache=true] {
  // Result cached by input hash
}
```

4. Sparse Representations

```
// Use sparse matrices for large hypergraphs
hypergraph large {
  representation: sparse
  backend: scipy.sparse
}
```

Debugging Level 7

Visualization

```
// Generate execution trace
workflow debug_mode {
  trace: true
  output: "execution_trace.json"
}

// Visualize hypergraph
visualize(hypergraph, format="graphviz")
```

Logging

```
// Detailed logging
workflow with_logging {
  log_level: DEBUG
  log_comonad_state: true
  log_probabilities: true
  log_aggregations: true
}
```

Profiling

```
// Profile performance
profile(workflow) {
  metrics: [tokens, time, cost]
  output: "profile.html"
}
```

Resources

Documentation

- **Main:** `/docs/DSL-COMPLEXITY-LEVEL-7.md` - **Research:** 3 foundation documents
(Comonads, Markov, Hypergraphs) - **Summary:** `/docs/LEVEL-7-SUMMARY.md`

Mathematical References

1. Uustalu & Vene (2005) - Comonadic dataflow 2. Fritz (2020) - Markov categories 3. Gallo et al. (1993) - Directed hypergraphs

Code Examples

- `/examples/level7_comonadic_loop.dsl` -
`/examples/level7_probabilistic_router.dsl` -
`/examples/level7_hypergraph_consensus.dsl` ---

Quick Start Template

```
// Level 7 Workflow Template
workflow my_level7_workflow {
  // 1. Define context (optional - for comonadic workflows)
  ctx = LLMContext {
    system_prompt: "...",
    history: [],
    temperature: 0.7,
    focus: initial_input
  }

  // 2. Define hypergraph structure (optional - for multi-way)
  hypergraph structure {
    vertices: [agent1, agent2, agent3, synthesizer]

    edge e1 = ({agent1, agent2, agent3}, {synthesizer})
    aggregation: "consensus"
  }

  // 3. Define probabilistic routing (optional - for adaptive)
  router = P(agent | difficulty)

  // 4. Execute workflow
  result = execute(structure, ctx)

  // 5. Optimize (optional - for self-tuning)
  optimize(workflow_params, objective="minimize_latency")

  return result
}
```

Common Questions

Q: When should I use Level 7 vs Level 6? A: Use Level 7 when you need infinite workflows, probabilistic execution, or multi-way dependencies. Use Level 6 for finite, deterministic, pairwise workflows. **Q: How do I handle #P-hard complexity?** A: Use approximation

algorithms: Monte Carlo sampling, Bayesian optimization, tensor decomposition. Exact computation is intractable. **Q: Can I mix levels?** A: Yes! Level 7 workflows can call Level 6 workflows as subroutines. Start simple and add complexity where needed. **Q: How do I debug infinite loops?** A: Add `.take(n)` or `.take_until(condition)` to limit iterations. Use `trace: true` for visibility. **Q: What's the performance overhead?** A: Typically <5% vs DAG systems if you use sparse representations and lazy evaluation. Tensor decomposition can provide 100x speedups. --- **This quick reference provides essential syntax, patterns, and guidance for working with Level 7. For comprehensive details, see the full documentation.** --- **Version: 1.0 Last Updated: 2025-10-19 Status: Complete**